

Bulut Bilişim Ödev Raporu

Hazırlayanlar:

Deniz Erdem ARAS	B221200014
Ahmet Timuçin UÇAN	B221200059
Metehan YILDIZ	B231200377

Sunum Videosu: <https://www.youtube.com/watch?v=uhfzDZ4CvI4>

Github Repo'su: <https://github.com/DenizErdemAras/Bulut-Bili-im-Proje/tree/main>

1. PROJE AMACI

Takımların görevlerini organize etmelerini sağlayan, sürükle-bırak destekli, özelleştirilebilir ve bulut tabanlı bir Kanban Yönetim Sistemi geliştirmektir.

Proje, modern DevOps prensiplerine uygun olarak "Konteynerize Mikroservis Mimarisi" temelinde tasarlanmıştır. Uygulama, ortam bağımsızlığı ve ölçeklenebilirlik sağlamak amacıyla Docker teknolojisi üzerine inşa edilmiş ve AWS bulut altyapısı üzerinde canlıya alınmıştır.

2. GÖREV DAĞILIMI

Proje, her biri sistemin kritik bir katmanından sorumlu olan üç kişilik bir mühendislik ekibi tarafından geliştirilmiştir. İş yükü, projenin uçtan uca tamamlanması için eşit ağırlıklı olarak dağıtılmıştır.

Ahmet Timuçin UÇAN: Sistem Mimarisi & DevOps

- Sorumluluk:** Konteyner orkestrasyonu ve Bulut Altyapısı.
- Gerçekleştirilen Görevler:**
 - Dockerizasyon:** PHP ve Apache için özel Dockerfile yazılarak gerekli sürücülerin sisteme entegre edilmesi.
 - Orkestrasyon:** docker-compose.yml dosyasının tasarlanması; Web ve Veritabanı servislerinin izole bir ağda çalıştırılması.
 - Bulut Dağıtımı:** AWS EC2 sunucusunun kurulumu ve Elastic IP yönetimi.
 - Veri Kalıcılığı:** Konteyner yaşam döngüsünden bağımsız veri saklama stratejisi kurgusu.

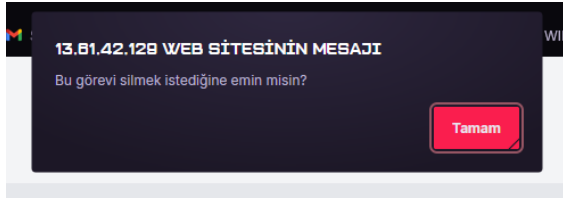
Metehan YILDIZ: Backend & Veritabanı

- Sorumluluk:** Sunucu taraflı iş mantığı, API Geliştirme ve Veri Tabanı Tasarımı.
- Gerçekleştirilen Görevler:**
 - Veritabanı Tasarımı:** İlişkisel veri modeli (todos ve categories tabloları) tasarımı ve UTF-8 (Türkçe karakter) uyumluluğunun sağlanması.

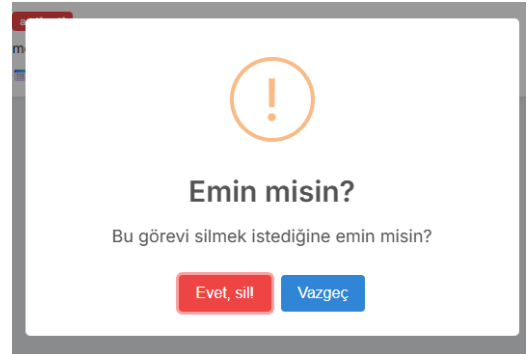
- **API Geliştirme:** Frontend ile haberleşen, RESTful prensiplerine uygun PHP API (api.php) kodlaması.
- **İş Mantığı:** Kategori yönetimi, görev statü güncellemeleri ve CRUD operasyonlarının backend tarafında güvenli şekilde işlenmesi.

Deniz Erdem ARAS: Frontend

- **Sorumluluk:** Kullanıcı Arayüzü, İstemci Mantığı ve Kullanıcı Deneyimi.
- **Gerçekleştirilen Görevler:**
 - **Arayüz Tasarımı:** HTML5 ve CSS3 kullanılarak responsive, modern bir Kanban tahtası tasarımı.
 - **Etkileşim:** JavaScript Drag & Drop API kullanılarak görevlerin sütunlar arası taşınma mantığının kodlanması.
 - **Asenkron İletişim:** Fetch API kullanılarak sayfa yenilenmeden veri alışverişi yapılması.
 - **UX Geliştirmeleri:** Akıllı tarih kontrolü (Gecikme uyarıları) ve görsel geri bildirim (SweetAlert2) sistemlerinin entegrasyonu.



SweetAlert2 öncesi



SweetAlert2 ile

3. GELİŞTİRME METODOLOJİSİ

Projede, yazılım hatalarını minimize etmek ve dağıtım sürekliliğini sağlamak amacıyla "Önce Yerel, Sonra Bulut" stratejisi izlenmiştir.

Adım 1: Yerel Ortamda Geliştirme

- **Süreç:** Veritabanı ve Web sunucusu servisleri yerel ortamda docker-compose ile ayağa kaldırılarak kodlama ve test işlemleri tamamlanmıştır.

Adım 2: Bulut Ortamına Geçiş

- **Süreç:** AWS EC2 üzerinde Ubuntu sunucusu hazırlanmış, yereldeki Docker imajları ve konfigürasyon dosyaları sunucuya taşınmıştır. Konteyner yapısı sayesinde, kodda hiçbir değişiklik yapılmadan proje saniyeler içinde canlıya alınmıştır.

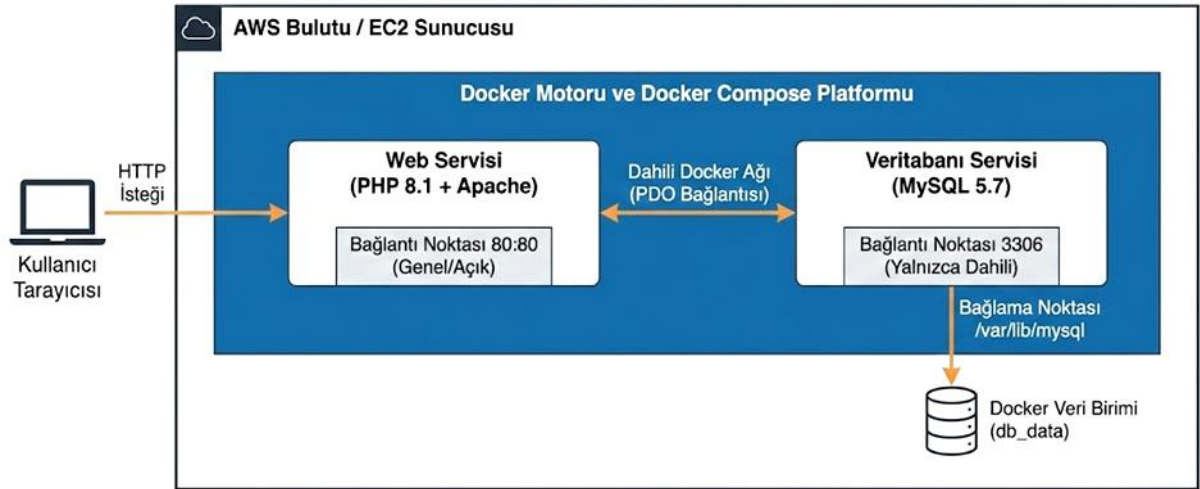
4. SİSTEM MİMARİSİ

Sistem, birbirinden izole edilmiş ancak birbiriyle haberleşen servislerden oluşan İki Katmanlı Mikroservis Mimarisine sahiptir.

1. **İstemci Katmanı:** Kullanıcı tarayıcısı (HTML/JS).
2. **Web Servisi:** 80 portundan yayın yapar. İstemciden gelen istekleri karşılar ve işler.
3. **Veritabanı Servisi:** 3306 portunda çalışır. Sadece Web servisinin erişebileceği dahili bir ağda bulunur.
4. **Veri Kalıcılığı:** db_data isimli Docker Volume, verilerin konteyner silinse bile sunucu diskinde kalıcı olmasını sağlar.

```
ubuntu@ip-172-31-40-147:~$ sudo docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
678141993a0c   kanban-projesi-web   "docker-php-entrypoi..." 3 hours ago   Up 3 hours   0.0.0.0:80->80/tcp, [::]:80->80/tcp   kanban-projesi-web-1
3fbd0fefe37    mysql:5.7          "docker-entrypoint.s..." 3 hours ago   Up 3 hours   3306/tcp, 33060/tcp                 kanban-projesi-db-1
```

Konteyner Tabanlı Web Uygulama Mimarisi (AWS ve Docker)



5. KULLANILAN TEKNOLOJİLER VE SEÇİM NEDENLERİ

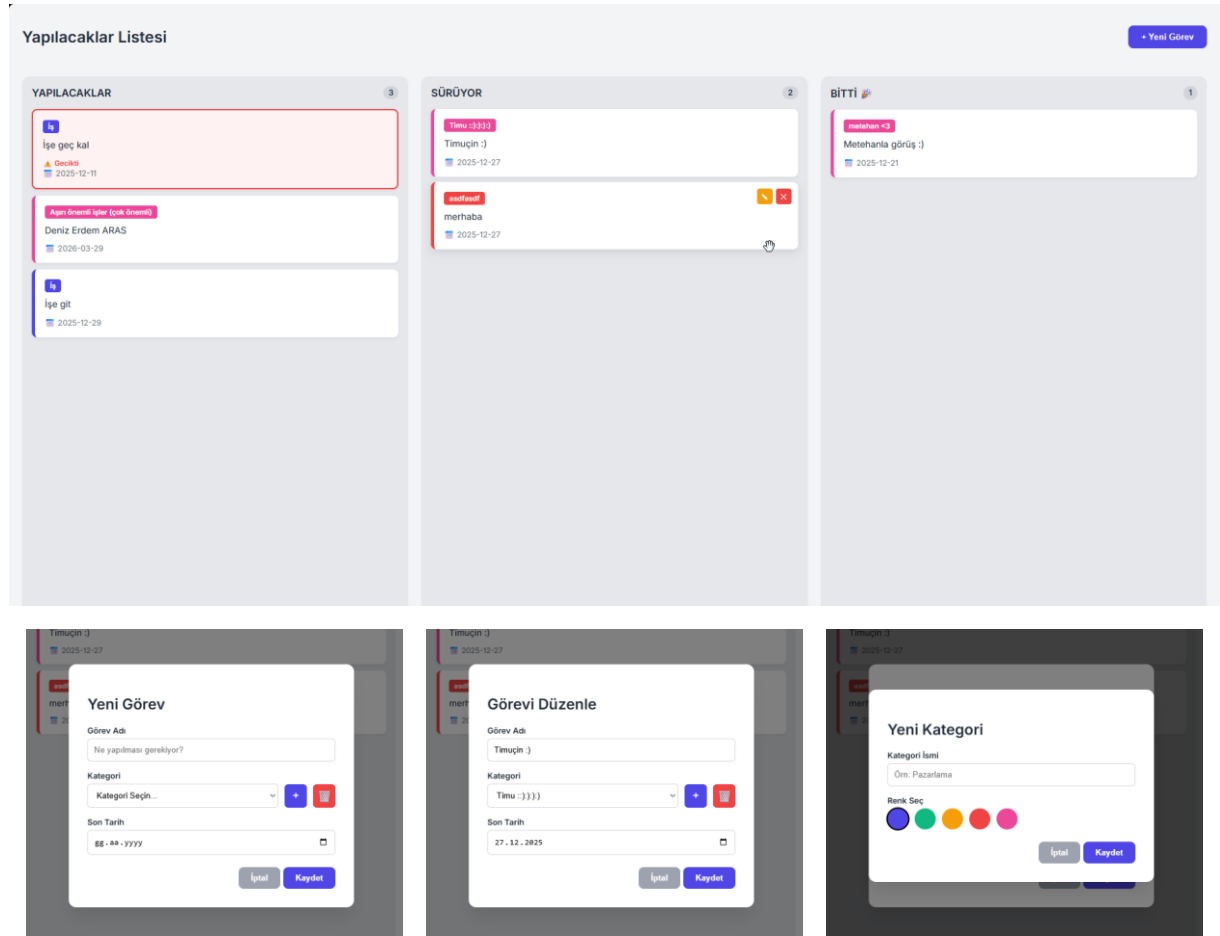
- **Amazon EC2:** Uygulamanın bağımlılıklarını izole etmek, kurulumu standartlaştırmak ve ortamdan bağımsız çalışabilmek için.
- **Docker & Docker Compose:** Uygulamanın bağımlılıklarını izole etmek, kurulumu standartlaştırmak ve ortamdan bağımsız çalışabilmek için.
- **PHP (Backend):** LAMP (Linux, Apache, MySQL, PHP) yığınının kararlılığı, Docker üzerindeki hafif performansı ve MySQL ile güçlü entegrasyonu nedeniyle tercih edilmiştir.

- **MySQL (Database):** İlişkisel veri yapısı (Görev-Kategori ilişkisi) ve veri bütünlüğü gereksinimleri için kullanılmıştır.
- **HTML5/JS/CSS3 (Frontend):** Tarayıcı uyumluluğu ve harici framework bağımlılığı olmadan hafif bir arayüz oluşturmak için seçilmiştir.

6. PROJE ÖZELLİKLERİ VE FONKSİYONLAR

Sistem, kullanıcı ihtiyaçları gözetilerek aşağıdaki temel özelliklerle donatılmıştır:

1. **Dinamik Kanban Yönetimi:** Görevler "Yapılacaklar", "Sürüyor" ve "Bitti" sütunları arasında sürükle-bırak yöntemiyle taşınabilir.
2. **Özelleştirilebilir Kategoriler:** Kullanıcılar standart kategorilere bağlı kalmadan, kendi isimlendirdikleri ve renk atadıkları kategoriler oluşturabilir.
3. **Akıllı Tarih Denetimi:** Sistem, son tarihi geçen görevleri otomatik algılar. Kart üzerinde kırmızı bir çerçeve ve "Gecikti" uyarısı göstererek kullanıcıyı görsel olarak uyarır.
4. **CRUD Modalları:** Görev ekleme, düzenleme ve silme işlemleri için kullanıcı dostu, sayfa yenilemesi gerektirmeyen formlar tasarlanmıştır.



7. ADIM ADIM UYGULAMA

Projenin bulut ortamında devreye alınması sürecinde izlenen teknik adımlar aşağıda kronolojik sırayla listelenmiştir.

Adım 1: AWS Bulut Altyapısının Hazırlanması

Projenin barındırılacağı sanal sunucu ortamı Amazon Web Services üzerinde aşağıdaki konfigürasyonlarla oluşturulmuştur:

1. EC2 Örneklemesi:

- **AMI:** Ubuntu Server 24.04 LTS (HVM), SSD Volume Type seçildi.
- **Instance Type:** t3.micro (Free Tier uyumlu) tercih edildi.
- **Key Pair:** Sunucuya SSH bağlantısı yapabilmek için .pem uzantılı bir anahtar çifti oluşturuldu ve indirildi.

2. Ağ ve Güvenlik Duvarı (Security Groups):

- Sunucunun dış dünyayla iletişim kurabilmesi için Inbound Rules şu şekilde yapılandırıldı:
 - **SSH (Port 22):** Yönetimsel erişim için.
 - **HTTP (Port 80):** Web uygulamasının son kullanıcıya sunulması için.

3. Statik IP Ataması (Elastic IP):

- Sunucu yeniden başlatıldığında IP adresinin değişmesini önlemek ve DNS tanımlarına sabit bir adres sunmak amacıyla AWS panelinden bir Elastic IP tahsis edildi ve oluşturulan EC2 örneği ile ilişkilendirildi.
-

Adım 2: Sunucu Ortamının Hazırlanması

SSH protokolü üzerinden sunucuya bağlanılarak gerekli paket güncellemeleri yapılmış ve konteyner motoru kurulmuştur.

1. Sistem Güncellemesi:

```
sudo apt-get update && sudo apt-get upgrade -y
```

2. Docker ve Docker Compose Kurulumu: Uygulamanın konteynerize çalışabilmesi için Docker Engine ve orkestrasyon aracı Docker Compose yüklendi:

```
sudo apt install docker.io -y
sudo curl -L
"https://github.com/docker/compose/releases/download/v2.20.2/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose
```

Adım 3: Proje Dizin Yapısı ve Konteyner Konfigürasyonu

Sunucu üzerinde /home/ubuntu/kanban-projesi dizini oluşturularak mikroservis mimarisini tanımlayan dosyalar hazırlandı.

1. **Dizin Yapısı:** Kaynak kodların düzenli durması için src klasörü oluşturuldu.
2. **Web Servisi İmaj Tanımı (Dockerfile):** PHP uygulamasının veritabanı ile konuşabilmesi için php:8.1-apache resmi imajı baz alınarak özel bir imaj tanımlandı ve docker-php-ext-install pdo pdo_mysql komutu ile gerekli sürücüler eklendi.
3. **Orkestrasyon Dosyası (docker-compose.yml):**
 - o **Web Servisi:** 80 portuna yönlendirildi ve kaynak kodlar (./src) volume olarak bağlandı.
 - o **Veritabanı Servisi:** mysql:5.7 imajı kullanıldı, root şifresi ve veritabanı ismi (tododb) environment değişkeni olarak tanımlandı.
 - o **Ağ:** Servislerin izole haberleşmesi için kanban-network tanımlandı.
 - o **Kalıcılık:** Veri kaybını önlemek için db_data volume'ü oluşturuldu.

Adım 4: Uygulama Kodlarının Entegrasyonu

Backend ve Frontend kodları sunucuya yerleştirildi.

1. **Backend (src/api.php):** Veritabanı bağlantı dizisinde (DSN) host=localhost yerine Docker servis adı olan host=db kullanılarak konteynerler arası DNS çözümlemesi sağlandı. PHP PDO yapısı ile CRUD işlemleri kodlandı.
2. **Frontend (src/index.html):** HTML5 Drag & Drop API kullanılarak sürükle-bırak mantığı ve Fetch API ile backend haberleşmesi sağlandı. Kullanıcı deneyimi için SweetAlert2 kütüphanesi CDN üzerinden projeye dahil edildi.

Adım 5: Servislerin Başlatılması ve Veritabanı Kurulumu

Tüm yapılandırma tamamlandıktan sonra uygulama canlıya alındı.

1. **Konteynerlerin Başlatılması:** Aşağıdaki komut ile imajlar derlendi (build) ve servisler arka planda (detached) çalıştırıldı:

```
sudo docker-compose up -d --build
```

2. **Veritabanı Şemasının Oluşturulması (Migration):** MySQL konteynerine komut satırından erişilerek ilk tablolar (todos ve categories) oluşturuldu ve Türkçe karakter desteği (utf8mb4) ayarlandı:

```
sudo docker-compose exec db mysql -u root -p<password> tododb -e "
CREATE TABLE IF NOT EXISTS todos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id VARCHAR(255) NOT NULL,
  text TEXT NOT NULL,
  category VARCHAR(50),
  color VARCHAR(20),
  due_date DATE,
  status VARCHAR(20) DEFAULT 'todo',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

CREATE TABLE IF NOT EXISTS categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id VARCHAR(255) NOT NULL,
  name VARCHAR(50) NOT NULL,
  color VARCHAR(20) DEFAULT '#4F46E5'
) CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
"
```

Adım 6: Test ve Doğrulama

Tarayıcı üzerinden sunucunun Public IP adresine (Elastic IP) erişim sağlanarak;

- Kanban tahtasının yüklendiği,
- Veritabanına kayıt eklenebildiği,
- Sürükle-bırak işleminin statüyü güncellediği doğrulanarak proje başarıyla tamamlandı.

8. Öğrenilen Dersler ve Olası İyileştirmeler

Proje geliştirme sürecinde elde edilen teknik kazanımlar ve uygulamanın gelecek sürümlerinde hayata geçirilmesi planlanan mimari genişlemeler aşağıda detaylandırılmıştır.

Öğrenilen Dersler

- **Ortam Eşitliği:** Geliştirme ve Üretim ortamlarının Docker ile birebir aynı tutulmasının, "dependency hell" (bağımlılık karmaşası) sorununu tamamen ortadan kaldırdığı deneyimlenmiştir.
- **Mikroservis İletişimi:** Konteynerler arası iletişimin localhost üzerinden değil, Docker'ın dahili DNS servisi üzerinden yapılmasının ağ izolasyonu ve güvenlik için kritik olduğu öğrenilmiştir.
- **Durum Yönetimi:** Ön yüzde yapılan sürükle-bırak işlemlerinin arka uçla senkronizasyonunda, asenkron yapının kullanıcı deneyimini kesintiye uğratmadan veri bütünlüğünü sağladığı görülmüştür.
-

Olası İyileştirmeler

Mevcut sistemin "Bireysel Kullanım" odağından çıkarılıp "Takım İşbirliği" platformuna dönüştürülmesi için aşağıdaki geliştirmeler planlanmıştır:

Ortak Çalışma Alanları ve Görev Grupları

Uygulamanın bir sonraki aşamasında, kullanıcıların sadece kendi görevlerini değil, dahil oldukları ekiplerin görevlerini de yönetebilecekleri bir sistem geliştirilebilir.

- **Grup Yönetimi:** Kullanıcılar yeni bir "Proje Grubu" oluşturabilecek veya benzersiz bir davet kodu/linki ile mevcut gruplara katılabileceklerdir.
- **Veritabanı İlişkisi:** Mevcut veritabanı şemasına groups ve user_groups tabloları eklenecektir. Kullanıcılar ve gruplar arasında Çoka-Çok ilişki kurularak, bir kullanıcının birden fazla gruba üye olması sağlanacaktır.
- **Bütünleşik Arayüz:**
 - Kullanıcı arayüzü dikey olarak iki ana bölüme ayrılacaktır.
 - **Üst Bölüm:** Kullanıcının sadece kendisinin görebildiği "Kişisel Yapılacaklar Listesi".
 - **Alt Bölüm:** Üyesi olunan grubun tüm üyeleriyle paylaşılan, gerçek zamanlı güncellenen "Ortak Proje Panosu".
- **Gerçek Zamanlı Senkronizasyon:** Grup içerisindeki bir üyenin görevi "Bitti" sütununa taşınması anında diğer üyelerin ekranına.

Kimlik Doğrulama

Mevcut localStorage tabanlı kimlik sistemi yerine, JWT (JSON Web Token) tabanlı güvenli bir Giriş/Kayıt Ol sistemi entegre edilerek kullanıcı verilerinin cihazlar arası taşınabilirliği sağlanacaktır.

9. SONUÇ

Bu proje kapsamında, modern yazılım mühendisliğinin temel taşları olan Bulut Bilişim ve Konteynerizasyon teknolojileri başarıyla uygulanmıştır. Ekip, mikroservis tabanlı bir mimari kurgulayarak ölçeklenebilir, taşınabilir ve güvenli bir ürün ortaya koymuştur.